

Reviewer Pack — Minimal Local Cohomology Rank and Communication Complexity R...

Entry 19ecc5c1c156 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 03:45:20 UTC

Minimal Local Cohomology Rank and Communication Complexity Rank Variance Ratio

Entry ID: 19ecc5c1c156 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 03:45:20 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Local Cohomology) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For all n -vertex graphs, the ratio of communication complexity rank variance to minimal local cohomology rank is bounded by a constant $C(n)$, i.e., $O(C(n)) \leq \text{CommRankVariance}(G) / \text{MinLocalCohomologyRank}(G)$.

Rationale (proposer's reasoning):

Local cohomology in algebraic geometry can provide insights into the structure of certain combinatorial objects, and its relationship with communication complexity could reveal new structural properties of graphs. This bridge might expose novel algorithms or lower bounds in communication complexity theory.

Taxonomy category: algebraic_geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 99d01badde259bce

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For all n -vertex graphs, if the ratio of communication complexity rank variance to minimal local cohomology rank exceeds a constant $C(n)$, then the conjecture is falsified; otherwise, it is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "communication complexity rank variance" AND "minimal local cohomology rank" - "algebraic geometry" AND "matrix rank" IN "communication complexity" - "graph theory" AND ("local cohomology" OR "commutative algebra") AND "communication complexity"

Top relevant hits considered: - [http://arxiv.org/abs/2512.13850v1] Characterization of projective varieties beyond varieties of minimal degree and del Pezzo varieties - [http://arxiv.org/abs/0910.5301v3] Using Elimination Theory to construct Rigid Matrices - [http://arxiv.org/abs/2601.16103v2] The Eisenbud-Goto conjecture for projectively normal varieties with mild singularities

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            if A[i][i] == 0:
                return None # Singular matrix
            for j in range(i + 1, m):
                factor = Fraction(A[j][i], A[i][i])
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
        rank = sum(1 for row in A if any(row))
        return rank

    def communication_complexity_rank_variance(G):
        n = len(G)
        ranks = []
        for i in range(n):
            for j in range(i + 1, n):
                subgraph = [[G[u][v] for v in range(j)] for u in range(i)]
                rank = gaussian_elimination(subgraph)
                if rank is not None:
                    ranks.append(rank)
        if not ranks:
```

```

        return 0
    max_rank = max(ranks)
    min_rank = min(ranks)
    variance = (max_rank - min_rank) ** 2 / len(ranks)
    return variance

def minimal_local_cohomology_rank(G):
    n = len(G)
    rank = sum(1 for row in G if any(row))
    return rank

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    comm_rank_variance = communication_complexity_rank_variance(G)
    min_local_cohomology_rank = minimal_local_cohomology_rank(G)
    if min_local_cohomology_rank == 0:
        continue
    ratio = Fraction(comm_rank_variance, min_local_cohomology_rank)
    results.append(ratio)

mean_ratio = sum(results) / len(results)
conjecture_holds = all(r <= mean_ratio for r in results)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Ratio of Communication Complexity Rank Variance to Minimal Local Cohomology Rank",
    "metric_value": float(mean_ratio),
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result["metric_value"])

    mean_ratio = sum(results) / len(results)
    support_fraction = sum(1 for r in results if r <= mean_ratio) / len(results)
    if support_fraction >= 0.8:
        result = f"SUPPORTED mean={mean_ratio} std={math.sqrt(sum((r - mean_ratio) ** 2 for r in results))}"
    elif any(r > mean_ratio for r in results):
        first_failing_seed = seeds[results.index(max(results))]
        result = f"FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}"
    else:
        result = "INCONCLUSIVE mapping_undefined"

    print(result)

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ce3fec44.py", line 82, in <module>
    trial_result = run_trial(seed)
                   ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ce3fec44.py", line 58, in run_trial
    comm_rank_variance = communication_complexity_rank_variance(G)
                       ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ce3fec44.py", line 39, in communication
    rank = gaussian_elimination(subgraph)
          ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ce3fec44.py", line 22, in gaussian_elimination
    m, n = len(A), len(A[0])
                   ~~~~
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means we cannot verify the conjecture's support or falsification. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12600
2	propose	ollama_remote	glm4:latest	0	0	12458
3	preregistration	ollama_remote	glm4:latest	0	0	8945
4	novelty	ollama_remote	glm4:latest	0	0	8379
5	novelty	ollama_remote	glm4:latest	0	0	9694
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15220
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12883
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9404
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9724
10	judge	ollama_remote	glm4:latest	0	0	23574

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 122882 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/19ecc5c1c156.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/19ecc5c1c156.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/19ecc5c1c156.tar.gz (if generated)